

FashionNest Full-Stack eCommerce - Interview QA Preparation Guide

This guide contains a comprehensive list of interview questions from basic to advanced regarding the FashionNest eCommerce web application.

Part 1: Basic Project Overview & Architecture

1. Can you give me an overview of your project?

Answer: FashionNest is a full-stack eCommerce web application. It allows users to browse products, filter by categories, add items to a shopping cart, and securely check out. I built the backend using Java (Servlets and JSP) and connected it to a MySQL database using JDBC. The entire application is containerized using Docker and deployed to the cloud on Render.

2. What design pattern did you use for this project?

Answer: The project strictly follows the **Model-View-Controller (MVC)** architectural pattern.

- * **Model:** Represented by Java POJO classes (e.g., `User`, `Product`, `Order`) and Data Access Objects (DAOs like `ProductDAO`, `UserDAO`) which handle all database interactions.
- * **View:** Represented by JSP (JavaServer Pages) which generate the dynamic HTML content sent to the browser.
- * **Controller:** Represented by Java Servlets (e.g., `CheckoutServlet`, `LoginServlet`) which intercept HTTP requests, process business logic via DAOs, and forward the results to the appropriate JSP views.

Part 2: Core Java, Servlets, and JSP

3. Explain the lifecycle of a Java Servlet in your application.

Answer: A Servlet's lifecycle consists of three main methods: 1. `init()`: Called once when the servlet is first initialized by the server (Tomcat) to allocate resources. 2. `service()`:

Called for every incoming HTTP request. It determines the HTTP method (GET, POST) and dispatches it to `doGet()` or `doPost()`. 3. `destroy()`: Called when the server shuts down or the servlet is removed, used to release resources. *In this project, my servlets (like `OrderServlet`) primarily override `doGet` to fetch data and `doPost` to submit data (like forms).*

4. How did you maintain state (e.g., the Shopping Cart) across multiple requests?

Answer: I used HTTP Sessions (`HttpSession` interface in Java). Because HTTP is a stateless protocol, when a user adds an item to the cart, the `CartServlet` retrieves the current session using `request.getSession()`. The cart (which is a `List<CartItem>`) is stored as a session attribute. This means the cart data is stored entirely in server memory while the user browses, reducing unnecessary database load until the user decides to checkout.

5. Why did you use JSP instead of just using raw Servlets or static HTML?

Answer: Servlets are great for handling Java logic but terrible for writing HTML because you have to use `out.println("<html>...")` for every line. JSP flips this: it allows writing standard HTML and embedding small snippets of Java code inside. It helps properly separate the View (JSP) from the Controller (Servlet).

Part 3: Database & JDBC

6. How does your Java backend communicate with the MySQL Database?

Answer: I used **JDBC (Java Database Connectivity)**. I have a singleton-like `DBConnection` class that loads the MySQL Driver (`Class.forName("com.mysql.cj.jdbc.Driver")`) and establishes a connection using `DriverManager.getConnection()`. All my DAO classes call this method to get a connection before executing queries.

7. How did you prevent SQL Injection in your project?

Answer: Throughout the application, I exclusively used `PreparedStatement` instead of standard `Statement`. `PreparedStatement` pre-compiles the SQL query on the database

side and safely escapes user inputs passed via `ps.setString()`. For example, in the `LoginServlet`, even if a user types malicious SQL as their password, it is treated strictly as a string literal, completely neutralizing SQL injection attacks.

8. Explain how the Checkout Process works at the Database level.

Answer: The checkout process is complex because it involves multiple tables (`orders` and `order_items`). In the `CheckoutServlet`: 1. I get a direct `Connection` and set `con.setAutoCommit(false)` to start a **transaction**. 2. I execute an `INSERT` into the `orders` table to generate the master order and retrieve the generated `order_id` using `Statement.RETURN_GENERATED_KEYS`. 3. I iterate through the session Cart, and for each item, I execute an `INSERT` into the `order_items` table using that `order_id`. 4. Finally, if everything succeeds, I call `con.commit()`. If any exception occurs (e.g., database crash), I catch it and call `con.rollback()`. This ensures Data Integrity—we never get an order without items, or items without an order.

Part 4: Docker & Cloud Deployment

9. Why did you use Docker, and what does your Dockerfile do?

Answer: Docker ensures the application runs exactly the same way on my local machine as it does on the cloud server. My `Dockerfile` does the following: 1. Pulls a base image containing Tomcat and JDK (`tomcat:10.1-jdk17-temurin`). 2. Installs `mysql-server` directly inside the container. 3. Copies my compiled application (`EcommerceApp.war`) into Tomcat's `webapps` folder. 4. Copies a custom shell script (`docker-entrypoint.sh`) and sets it to run when the container starts.

10. We saw a fix regarding a `dockeruser` in your deployment logs. What was that issue and how did you debug it?

Answer: During deployment on Render, Tomcat started successfully, but the website failed to load products. By looking at the logs, I realized the database was "empty" to the Java app. My Java `DBConnection` class was configured to log in to MySQL using `dockeruser`. However, my initialization script only set up the `root` user. The Java app was getting an "Access Denied" error. I fixed this by modifying my `docker-entrypoint.sh` to execute

```
CREATE USER 'dockeruser' and GRANT ALL PRIVILEGES ON ecommerce.* TO 'dockeruser',
```

 which immediately resolved the connection issue in production.

11. Why do you have a "Wait for MySQL to be ready" loop in your `docker-entrypoint.sh` ?

Answer: In my Docker setup, both Tomcat (the app server) and MySQL (the database) start in the same container. MySQL takes a few seconds to initialize its internal tables and socket. If I run the `ecommerce.sql` data import immediately, it fails because MySQL isn't ready to accept connections yet. The loop pings `mysql -e "SELECT 1"` every second until it succeeds, ensuring the database is fully booted before importing the schema and starting Tomcat.

Part 5: Advanced Scenarios

12. What happens if 1000 users add items to their cart at the exact same time?

Answer: Since the cart is managed via `HttpSession`, 1000 users mean 1000 separate session objects in Tomcat's memory. Tomcat handles multi-threading gracefully, allocating a separate thread for each incoming request. The database is entirely bypassed during the "add to cart" phase. However, if traffic scaled extremely high, Tomcat might run out of JVM heap memory due to too many sessions. In a real-world enterprise scenario, I would migrate the session storage to a distributed cache like Redis.

13. If you were to upgrade this project, what modern framework would you use?

Answer: I would migrate the backend from raw Servlets to **Spring Boot**. Spring Boot eliminates the heavy boilerplate of JDBC by providing Spring Data JPA (Hibernate) for object-relational mapping. It also provides an embedded Tomcat server, simple REST API creation capabilities (for connecting a React/Angular frontend), and easier dependency injection, which would make the codebase much cleaner and highly scalable.